

## FASE 1 LOGIKA PEMROGRAMAN

Nama : Muhammad Zarif Falah  
NIM : 26/579453/TK/66007  
Kelompok : Calcite  
Prodi : Teknologi Informasi

### PROBLEM 1: THE LAST ASTRONAUT

#### Pseudocode:

#### MULAI

**Masukkan** N

**Masukkan** K

**Buat** daftar astronot dari 1 sampai N

**Buat** daftar untuk menyimpan urutan astronot yang gugur

**Selama** jumlah astronot lebih dari 1:

// Tentukan posisi astronot yang akan gugur:  
posisi = (posisi + K - 1) MOD jumlah astronot

**Ambil** nomor astronot pada posisi tersebut

**Simpan** nomor tersebut sebagai astronot yang gugur

**Hapus** astronot tersebut dari daftar

Kurangi jumlah astronot sebanyak 1

**Jika** nomor astronot yang gugur genap:

K bertambah 2

**Jika** nomor astronot yang gugur ganjil:

K berkurang 1

**Jika** K kurang dari 2:

K menjadi 2

Lanjutkan perhitungan dari astronot setelah yang gugur

**Tampilkan** semua astronot yang gugur secara berurutan

**Tampilkan** astronot yang tersisa sebagai pemenang

SELESAI

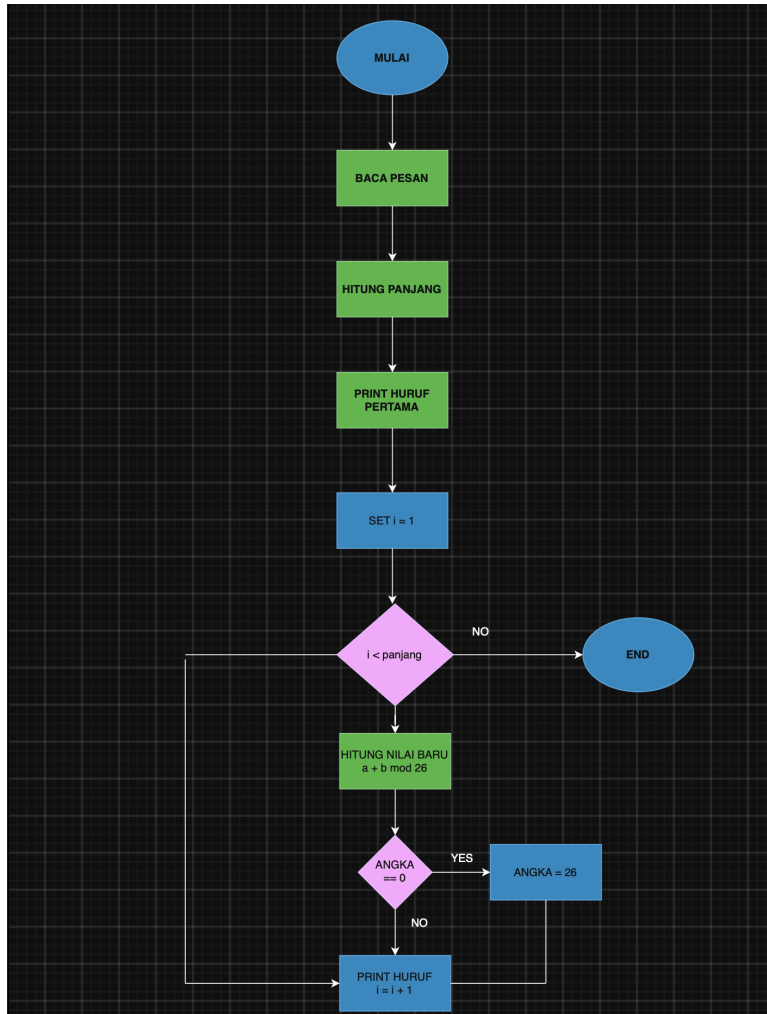
### Penjelasan Logika (Problem 1)

Program dimulai dengan membaca  $N$  (jumlah astronot) dan  $K$  (nilai  $K$  awal), lalu membuat array dinamis astronot sebesar  $N$  menggunakan `new int[n]` dan mengisinya dengan nomor 1 sampai  $n$  lewat perulangan `for`. Dua variabel penting yang menjaga status simulasi adalah *sisa* (jumlah astronot yang masih hidup, awalnya sama dengan  $n$ ) dan *pos* (index tempat hitungan dimulai, awalnya 0 yang berarti astronot nomor 1). Seluruh proses eliminasi berjalan di dalam `while (sisa > 1)`, yang terus berulang selama astronot yang tersisa masih lebih dari satu orang. Di setiap putaran, baris `pos = (pos + k - 1) % sisa`; adalah inti dari simulasi hitungan melingkar — ini menggeser posisi sejauh  $k$  langkah dari posisi sekarang, dan operasi modulo  $\%$  *sisa* memastikan kalau hitungannya lewat dari astronot terakhir, dia otomatis "muter balik" ke awal lingkaran lagi.

Setelah posisi yang tereliminasi ditemukan, nomor astronot di posisi tersebut (`astronot[pos]`) disimpan ke variabel `nomorGugur`, lalu dicatat ke dalam array `urutanEliminasi` sebagai riwayat, dan `totalEliminasi` ditambah satu. Bagian `for (int i = pos; i < sisa - 1; i++) astronot[i] = astronot[i+1]`; penghapusan dilakukan dengan menggeser semua elemen setelah posisi `pos` satu langkah ke kiri, lalu `sisa--` mengecilkan ukuran lingkaran yang efektif. Setelah astronot gugur, kondisi `if (nomorGugur % 2 == 0) k += 2; else k -= 1;` menerapkan aturan perubahan nilai  $K$  sesuai ganjil-genapnya nomor yang tereliminasi, dan baris `if (k < 2) k = 2;` menjaga  $K$  supaya tidak pernah turun di bawah batas minimalnya. Terakhir, `pos = pos % sisa`; diperlukan karena setelah elemen-elemen digeser, `pos` secara otomatis sudah menunjuk ke astronot berikutnya di posisi yang sama — tinggal dipastikan saja nilainya tidak melebihi jumlah astronot yang tersisa, supaya tidak terjadi akses di luar batas array ketika lingkaran semakin mengecil. Perulangan ini terus berjalan sampai *sisa* tinggal 1, dan astronot yang bertahan di `astronot[0]` itulah yang dicetak sebagai pemenang di akhir program.

## PROBLEM 2: ALIEN-IN-THE-MIDDLE

### Flowchart



### Penjelasan Logika:

Proses dimulai dari **Start** lalu masuk ke tahap **Baca Pesan** untuk membaca input pesan dari pengguna, dilanjutkan dengan **hitung panjang** yang menghitung panjang pesan tersebut lewat perulangan sampai menemukan karakter kosong (tanda akhir kata). Setelah panjangnya

diketahui, alur langsung menuju **Print huruf pertama**, huruf pertama pesan dicetak apa adanya tanpa diubah, karena memang belum ada huruf sebelumnya yang bisa dijadikan acuan perhitungan. Barulah setelah itu variabel *i* diset ke 1 (**Set i = 1**) sebagai penanda posisi huruf kedua, tempat proses penyandian yang sesungguhnya baru dimulai.

Dari titik ini, flowchart masuk ke bagian perulangan yang dikendalikan oleh kotak keputusan "**i < panjang?**". Selama kondisi ini bernilai *Ya*, artinya masih ada huruf yang perlu diproses, sehingga alur masuk ke kotak **hitung nilai baru**, tempat nilai huruf sekarang dijumlahkan dengan nilai asli huruf sebelumnya, lalu dibungkus dengan modulo 26 supaya tidak melebihi batas alfabet. Hasil perhitungan ini kemudian diperiksa lagi lewat keputusan kedua, "**angka == 0?**" — inilah bagian yang menangani kasus khusus ketika penjumlahan pas kelipatan 26: kalau hasilnya 0, nilai dipaksa menjadi 26 (kotak **angka = 26**) supaya tetap terpetakan ke huruf Z, bukan karakter yang salah. Setelah nilai akhirnya pasti benar (baik lewat cabang *Ya* maupun langsung dari cabang *Tidak*), kedua jalur bertemu di kotak **Print Huruf**, yang mencetak huruf hasil sandi sekaligus menyimpan nilai asli huruf saat ini untuk dipakai sebagai acuan huruf berikutnya, lalu menambah *i* sebanyak satu. Dari sini, alur kembali (panah "next i") ke kotak keputusan "**i < panjang?**" untuk mengulang proses yang sama pada huruf berikutnya, dan siklus ini terus berjalan sampai seluruh huruf habis diproses — barulah kondisi menjadi *Tidak* dan program berhenti di **End**.